EX No:08

Date:

Implementation of LOOCV for Weekly Dataset

8.In Sections 5.1.2 and 5.1.3, we saw that the cross_validate() function can be used in order to compute the LOOCV test error estimate. Alternatively, one could compute those quantities using just sm.GLM() and the predict() method of the fitted model within a for loop. You will now take this approach in order to compute the LOOCV error for a simple logistic regression model on the Weekly data set. Recall that in the context of classification problems, the LOOCV error is given in (5.4).

LOAD LIBRARY:

```
import pandas as pd
import statsmodels.api as sm
from ISLP import load_data
```

LOAD DATA:

```
Weekly = load_data("Weekly")
Weekly["Up"] = (Weekly["Direction"] == "Up").astype(int)
Weekly.head()
```

OUTPUT:

	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today	Direction	Up
0	1990	0.816	1.572	-3.936	-0.229	-3.484	0.154976	-0.270	Down	0
1	1990	-0.270	0.816	1.572	-3.936	-0.229	0.148574	-2.576	Down	0
2	1990	-2.576	-0.270	0.816	1.572	-3.936	0.159837	3.514	Up	1
3	1990	3.514	-2.576	-0.270	0.816	1.572	0.161630	0.712	Up	1
4	1990	0.712	3.514	-2.576	-0.270	0.816	0.153728	1.178	Up	1

(a) Fit a logistic regression model that predicts Direction using Lag1 and Lag2.

CODE:

```
X = sm.add_constant(Weekly[["Lag1", "Lag2"]])
y = Weekly["Up"]
model_a = sm.GLM(y, X, family=sm.families.Binomial()).fit()
print(model_a.summary())
```

OUTPUT:

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                     Up   No. Observations:                 1089
Model:                            GLM   Df Residuals:                     1086
Model Family:                Binomial   Df Model:                            2
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -744.11
Date:                Mon, 12 Oct 2026   Deviance:                       1488.2
Time:                        10:27:28   Pearson chi2:                 1.09e+03
No. Iterations:                     4   Pseudo R-squ. (CS):          0.007303
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.2212      0.061      3.599      0.000       0.101       0.342
Lag1          -0.0387      0.026     -1.477      0.140      -0.090       0.013
Lag2           0.0602      0.027      2.270      0.023       0.008       0.112
==============================================================================
```

(b) Fit a logistic regression model that predicts Direction using Lag1 and Lag2 using all but the first observation.

CODE:

```
X_train = X.iloc[1:]
y_train = y.iloc[1:]
model_b = sm.GLM(y_train,X_train,family=sm.families.Binomial()).fit()
print(model_b.summary())
```

OUTPUT:

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                     Up   No. Observations:                 1088
Model:                            GLM   Df Residuals:                     1085
Model Family:                Binomial   Df Model:                            2
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -743.26
Date:                Mon, 12 Oct 2026   Deviance:                       1486.5
Time:                        10:28:12   Pearson chi2:                 1.09e+03
No. Iterations:                     4   Pseudo R-squ. (CS):          0.007373
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.2232      0.061      3.630      0.000       0.103       0.344
Lag1          -0.0384      0.026     -1.466      0.143      -0.090       0.013
Lag2           0.0608      0.027      2.291      0.022       0.009       0.113
==============================================================================
```

(c) Use the model from (b) to predict the direction of the first observation. You can do this by predicting that the first observation will go up if P (Direction = "Up"|Lag1, Lag2) > 0.5. Was this observation correctly classified?

CODE:

```
X_test = X.iloc[[0]]
probability = model_b.predict(X_test).iloc[0]
prediction = "Up" if probability > 0.5 else "Down"
actual = Weekly["Direction"].iloc[0]
print("Probability of Up:", probability)
print("Predicted Direction:", prediction)
print("Actual Direction:", actual)

if prediction == actual:
    print("Correctly classified")
else:
    print("Incorrectly classified")
```

OUTPUT:

```
Probability of Up: 0.571392320520443
Predicted Direction: Up
Actual Direction: Down
Incorrectly classified
```

(d) Write a for loop from i = 1 to i = n, where n is the number of observations in the data set, that performs each of the following steps:

i. Fit a logistic regression model using all but the ith observation to predict Direction using Lag1 and Lag2. ii. Compute the posterior probability of the market moving up for the ith observation.

iii. Use the posterior probability for the ith observation in order to predict whether or not the market moves up.

iv. Determine whether or not an error was made in predicting the direction for the ith observation. If an error was made, then indicate this as a 1, and otherwise indicate it as a 0.

CODE:

```
n = len(Weekly)
errors = [ ]
for i in range(n):


    X_train = X.drop(X.index[i])
    y_train = y.drop(y.index[i])
    model = sm.GLM(
        y_train, X_train,family=sm.families.Binomial()).fit()

    X_test = X.iloc[[i]]
    probability = model.predict(X_test).iloc[0]
    predicted = 1 if probability > 0.5 else 0


    actual = y.iloc[i]

    error = 1 if predicted != actual else 0

    errors.append(error)

print("First 10 errors:", errors[:10])
print("Number of observations:", n)
print("Number of errors:", sum(errors))
```

OUTPUT:

```
First 10 errors: [1, 1, 0, 1, 0, 1, 0, 0, 0, 1]
Number of observations: 1089
Number of errors: 490
```

(e) Take the average of the n numbers obtained in (d)iv in order to obtain the LOOCV estimate for the test error. Comment on the results

CODE:

```
loocv_error = sum(errors) / n

print("LOOCV Test Error:", loocv  error)
print("LOOCV Test Error (%):", loocv_error * 100)
print("LOOCV Accuracy (%):", (1 - loocv_error) * 100)
```

OUTPUT:

```
LOOCV Test Error: 0.44995408631772266
LOOCV Test Error (%): 44.995408631772264
LOOCV Accuracy (%): 55.00459136822773
```

INTERPRETATION:

- (a) In the logistic regression of Direction on Lag1 and Lag2, Lag2 is significant (0.0602, p = 0.023) while Lag1 is not (-0.0387, p = 0.140). The pseudo R-squared is only 0.007, so the model has very little explanatory power.
- (b), (c) Refitting without the first observation and predicting it gives P(Up) = 0.571, so the model predicts Up, but the actual direction was Down: the first observation is misclassified.
- (d) Repeating this leave-one-out procedure for all 1,089 weeks produced 490 errors; the first ten errors are [1, 1, 0, 1, 0, 1, 0, 0, 0, 1].
- (e) The LOOCV test error estimate is 490 / 1,089 = 44.99%, i.e. an accuracy of 55.0%. Always predicting Up would be right 605 / 1,089 = 55.6% of the time, so the model is no better than that naive rule and has essentially no useful predictive power for weekly direction.
- LOOCV uses almost all of the data for every fit, so its estimate is nearly unbiased and not dependent on a random split, but it needs 1,089 model fits.